



LegalAid

AI-Assisted Public Legal Services Platform

SYSTEM DESIGN, AI INTEGRATION & REFLECTION REPORT

PROJECT	LegalAid — AI-Assisted Public Legal Services Platform
TEAM MEMBERS	Kulsoom Jawed, Ammar Amin Khowaja, Anchal Bai
COURSE	Web Technologies — Course Project
CLIENT SCENARIO	Justice Connect (non-profit legal aid organisation)
DOCUMENT COVERS	Deliverable 1 — System Design Document · Deliverable 4 — AI Integration Report · Deliverable 5 — Written Reflection
COMPANION ARTEFACTS	Source repository (NestJS backend + React frontend), submitted alongside this document
DATE	15 July 2026

Table of Contents

Deliverable 1 — System Design Document.....

3

1. Purpose & Scope

3

2. High-Level Architecture

3

3. Entity–Relationship Diagram

3

4. Case Lifecycle — Guarded State Machine

4

5. API Contract.....

5

6. React Component Tree & Role Visibility.....

7

7. AI Integration Architecture.....

8

Deliverable 4 — AI Integration Report

10

1. Overview.....

10

2–5. AI Features 1–4 (prompts, output handling, fallback).....

10

6. Context Management Strategy

13

7. Fallback / Degradation Behaviour Summary

13

8. Prompt Iteration Log

13

9. Data Privacy Implications

13

Deliverable 5 — Reflection Report.....

14

1. Centralising all LLM traffic behind a single AI proxy.....

14

2. Fire-and-forget asynchronous AI triage.....

15

3. Enforcing data integrity in the service layer.....

15

Closing note.....

16

Technology stack	React · NestJS · PostgreSQL · Gemini LLM API · Socket.IO WebSocket · JWT Auth
Track	Intermediate Web Technologies · Full-Stack Track
Client scenario	Justice Connect — 42 volunteers, ~500 citizen enquiries/week across 3 cities
Contents	Deliverable 1 — System Design · Deliverable 4 — AI Integration Report · Deliverable 5 — Reflection Report

DELIVERABLE 1

System Design Document

1. Purpose & Scope

This document describes the as-built system design of LegalAid, extracted directly from the submitted source repository (legalaid-backend and legalaid-frontend). It covers the entity-relationship model, the REST API contract, the React component tree with role visibility, and the AI integration architecture, satisfying the four required sections of the System Design Document deliverable.

2. High-Level Architecture

LegalAid is a three-tier system: a React single-page application, a modular NestJS API, and a PostgreSQL database, with a dedicated AI proxy module acting as the sole gateway to the external LLM provider. Real-time updates travel over a Socket.IO gateway layered alongside the REST API, so citizens and volunteers see status changes and reminders without polling.

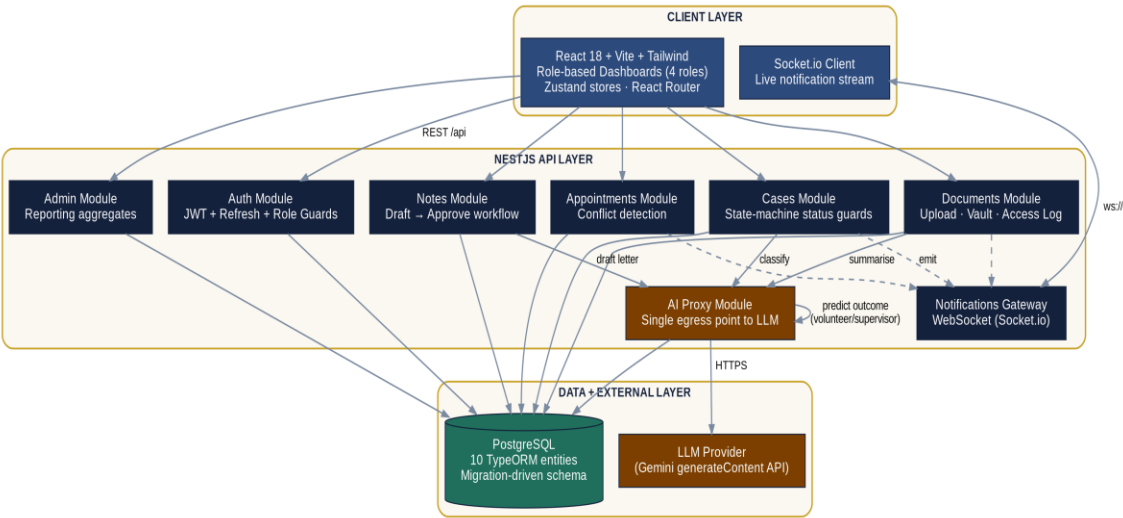


Figure 1 — Layered architecture: client, NestJS module layer, and data/external layer.

Nine feature modules sit behind the API gateway: Auth, Cases, Documents, Appointments, Notes, Notifications, Admin, Users, and the AI Proxy. Each module owns its own TypeORM entities, DTOs, and a Nest controller guarded by JwtAuthGuard and a role-based RolesGuard, so route-level authorisation is enforced consistently across the whole surface rather than re-implemented per endpoint.

3. Entity-Relationship Diagram

The schema is defined in code as ten TypeORM entities (User, Case, CaseStatusLog, Document, DocumentAccessLog, CaseNote, Appointment, Availability, Notification, and AiInteraction) and is version-controlled through a TypeORM migration (InitialSchema) rather than relying on runtime schema sync — the data source is configured with `synchronize: false`, so the schema only changes through reviewable migration files.

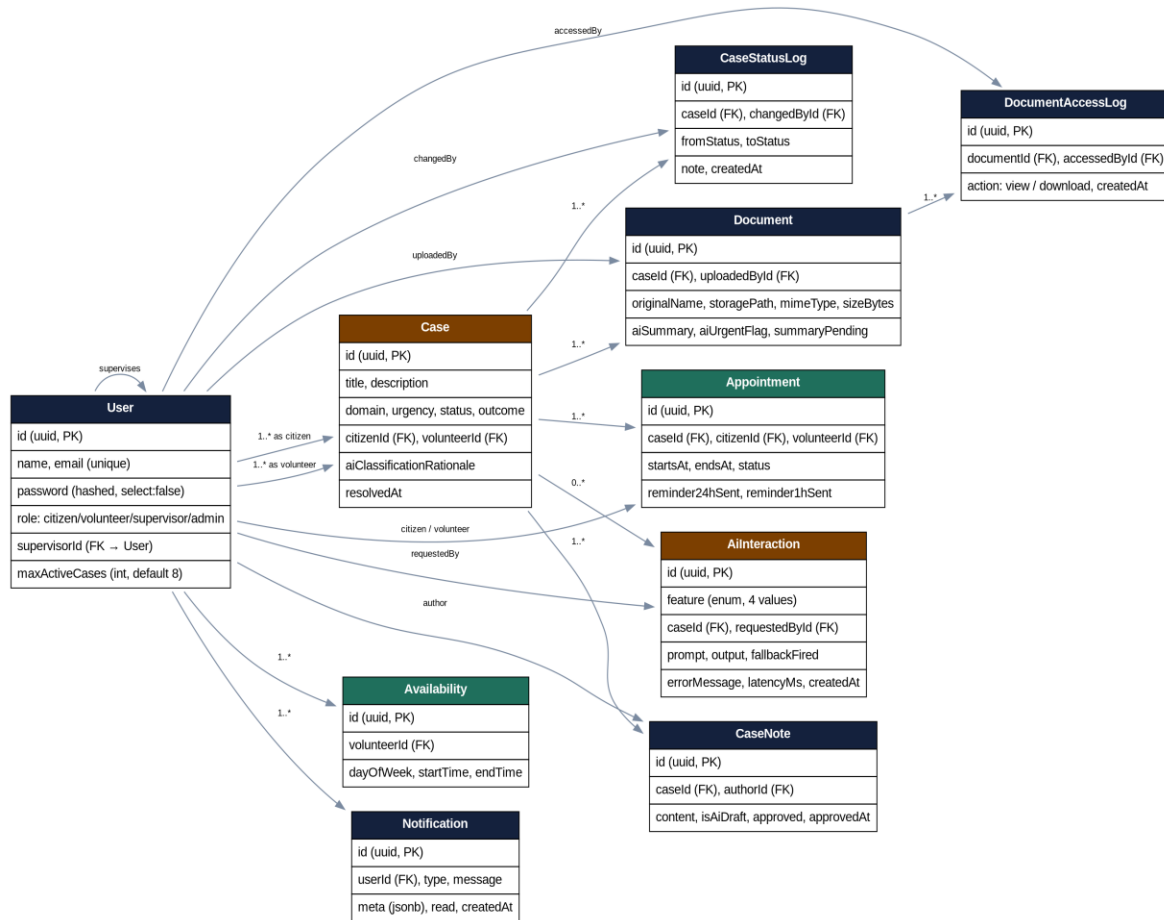


Figure 2 — Entity-relationship diagram covering all ten persisted entities and their foreign-key relationships.

3.1 Notable modelling decisions

- Every status change on a Case is append-only in CaseStatusLog (fromStatus, toStatus, changedBy, note) — this is the audit trail the grant-reporting requirement in the brief calls for, not just a status column.
- DocumentAccessLog records every view/download against a document, independent of the AI summary, to satisfy the “document access is logged for audit purposes” requirement.
- AiInteraction is a first-class entity (feature, prompt, output, fallbackFired, errorMessage, latencyMs) — every AI call, successful or not, is persisted, which is what makes the AI Integration Report's fallback analysis possible.
- User.maxActiveCases and a supervisor self-reference (supervisorId) let the Cases module enforce a volunteer capacity ceiling at assignment time (see §6).

4. Case Lifecycle — Guarded State Machine

Case status is not a free-text field: `CASE_STATUS_TRANSITIONS` is an explicit adjacency map enforced in `CasesService.updateStatus()`. Any request for a status not in the current state's allow-list is rejected with a 400 before it reaches the database, which is what “guarded transitions” in the brief means concretely.

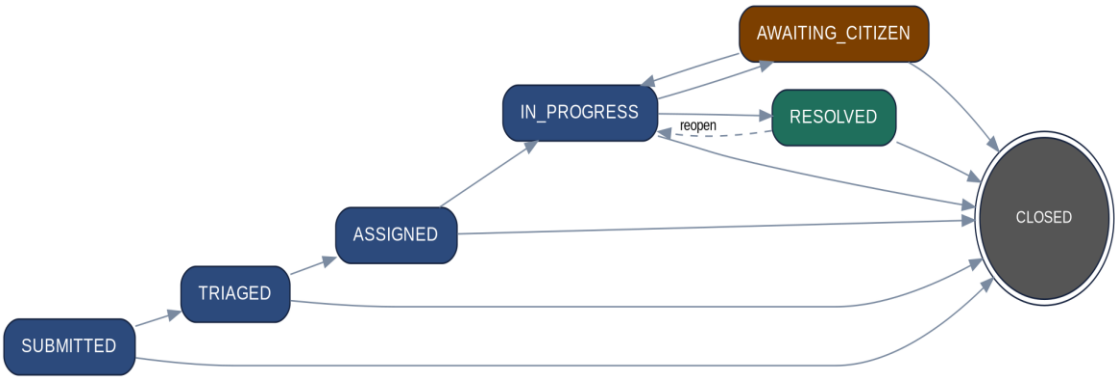


Figure 3 — Case status transition graph, generated from `CASE_STATUS_TRANSITIONS` in `case.enums.ts`.

Status	Allowed next states	Who can trigger it
submitted	triaged, closed	System (AI auto-triage) or manual classification
triaged	assigned, closed	Supervisor / Admin (assign)
assigned	in_progress, closed	Assigned volunteer, Supervisor, Admin
in_progress	awaiting_citizen, resolved, closed	Assigned volunteer, Supervisor, Admin
awaiting_citizen	in_progress, closed	Assigned volunteer, Supervisor, Admin
resolved	closed, in_progress (reopen)	Assigned volunteer, Supervisor, Admin
closed	— (terminal)	—

Volunteers may only act on cases they are assigned to (`found.volunteerId !== user.userId` is rejected with `ForbiddenException`); Supervisors and Admins can act on any case. Case assignment additionally enforces a per-volunteer capacity limit: `CasesService.assign()` counts the volunteer's non-closed cases and rejects the assignment once it reaches `User.maxActiveCases`, returning a message naming the volunteer and the limit so a supervisor can reassign or raise it.

5. API Contract

All protected routes require a Bearer JWT and are additionally restricted with `@Roles()`. The table below reflects the controllers as implemented; Swagger (`@nestjs/swagger`) is generated from the same decorators at runtime, so `/api/docs` and this table cannot drift apart.

5.1 Auth

Method	Endpoint	Roles	Notes
POST	/auth/register	Public	Creates a citizen account
POST	/auth/login	Public	Returns access + refresh JWT pair
POST	/auth/refresh	Authenticated (refresh token)	Rotates the access token
GET	/auth/me	Authenticated	Returns the current user profile

5.2 Cases

Method	Endpoint	Roles	Notes
POST	/cases	Citizen	Creates a case; fires async AI classification
GET	/cases	All (scoped)	Citizen sees own; Volunteer sees assigned + unassigned; Sup./Admin see all. Supports domain/status/volunteerId/page/limit
GET	/cases/stats/status-breakdown	Citizen	Status counts for the citizen's own cases
GET	/cases/:id	Owner, assigned volunteer, Sup./Admin	Includes citizen, volunteer, statusLogs, documents, notes
PATCH	/cases/:id/classify	Owner or Sup./Admin	Manual classification — the AI-unavailable fallback path
PATCH	/cases/:id/status	Volunteer (own), Sup., Admin	Guarded by CASE_STATUS_TRANSITIONS
PATCH	/cases/:id/assign	Sup., Admin	Enforces volunteer capacity limit
PATCH	/cases/:id/outcome	Volunteer (own), Sup., Admin	Sets won/settled/referred/withdrawn/unresolved
DELETE	/cases/:id	Admin, Citizen (own)	Removes a case

5.3 Documents

Method	Endpoint	Roles	Notes
POST	/cases/:caseId/documents	All roles	Multier disk storage, 10 MB limit, UUID filename
GET	/cases/:caseId/documents	Scoped to case access	Lists documents + AI summaries
GET	/documents/:id/download	Scoped to case access	Streams file; writes a DocumentAccessLog row
GET	/documents/:id/access-logs	Supervisor, Admin	Audit trail for a document
DELETE	/documents/:id	Scoped to case access	Removes a document

5.4 Notes & AI Letter Drafting

Method	Endpoint	Roles	Notes
POST	/cases/:caseId/notes	Volunteer	Rough note or AI-drafted letter (isAiDraft flag)
GET	/cases/:caseId/notes	Volunteer, Supervisor	Full note history for a case
PATCH	/notes/:id/approve	Volunteer	Explicit approval gate before a letter is sent

5.5 Appointments

Method	Endpoint	Roles	Notes
POST	/appointments	Citizen	Validates availability coverage + conflict window
GET	/appointments	All (scoped)	Citizen's own or volunteer's booked slots
PATCH	/appointments/:id	Volunteer	Confirm / reschedule / cancel
POST	/availability	Volunteer	Recurring weekly slot (day, start, end)
GET	/availability/:volunteerId	All	Public weekly availability for booking UI
DELETE	/availability/:id	Volunteer (own)	Removes a slot

5.6 AI Proxy, Notifications, Admin & Users

Method	Endpoint	Roles	Notes
POST	/ai/predict-outcome	Volunteer, Sup., Admin	Bonus Feature 4; never exposed to citizens
GET	/notifications	Authenticated	Latest 50 notifications for the caller
PATCH	/notifications/:id/read	Authenticated	Marks a notification read
GET	/admin/stats/*	Admin	volume, volume-by-domain/-volunteer/-month, resolution-time, outcomes, utilisation, status-breakdown
GET / PATCH / DELETE	/admin/users	Admin	User management
GET	/users/volunteers	Authenticated	Volunteer picker for assignment/booking
POST	/users/invite	Admin	Creates a pre-provisioned account
PATCH	/users/:id/capacity	Sup., Admin	Adjusts maxActiveCases

The WebSocket gateway (Socket.IO) is not a REST resource but is part of the contract: clients call join with their JWT to be placed in a user:<id> room and a role:<role> room, after which the server pushes case:status_changed, case:assigned,document:uploaded,appointment:reminder,appointment:requested,appointment:status_changed, case:high_urgency and case:new events without the client polling.

6. React Component Tree & Role Visibility

The frontend is a single Vite React app with one router (App.tsx) branching into four role-scoped route groups, each wrapped in RoleRoute (role check) and a shared RoleLayout (Sidebar + Topbar). Zustand stores (caseStore, notificationStore) hold cross-page state; useSocket connects the WebSocket gateway once per session.

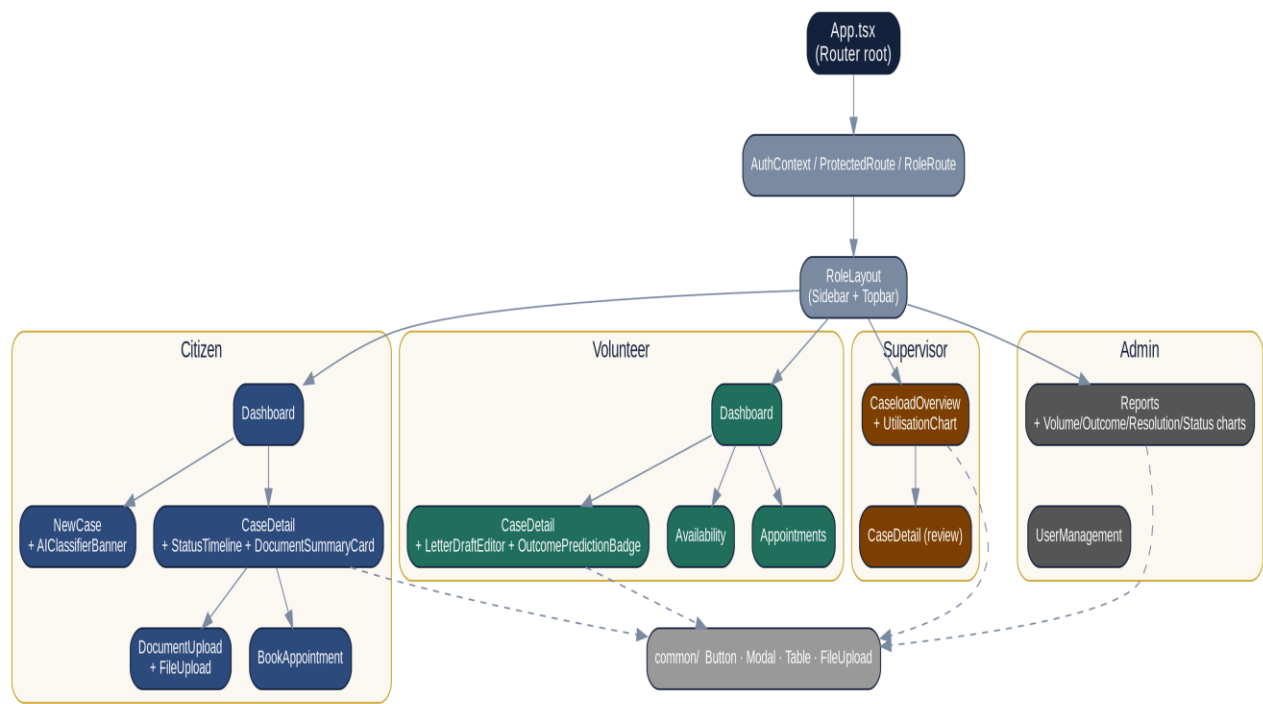


Figure 4 — Component tree grouped by role, showing where AI-powered components and chart components are mounted.

Role	Routes	AI-powered components
Citizen	/citizen, /citizen/new-case, /citizen/cases/:id, /documents, /book	AIClassifierBanner, DocumentSummaryCard
Volunteer	/volunteer, /volunteer/cases/:id, /availability, /appointments	LetterDraftEditor, DocumentSummaryCard
Supervisor	/supervisor, /supervisor/cases/:id	OutcomePredictionBadge, UtilisationChart
Admin	/admin (Reports), /admin/users	VolumeChart, OutcomeChart, ResolutionTimeChart, StatusBreakdownChart

7. AI Integration Architecture

Every AI feature is invoked from the frontend through the backend only — there is no LLM API key or endpoint reachable from React. `AiProxyService.call()` is the single choke point: it builds the prompt, checks for `GEMINI_API_KEY`, performs the HTTPS request to Gemini's `generateContent` endpoint, parses the response, and unconditionally logs an `AiInteraction` row, whether the call succeeded, failed, or never left the server because the key was absent.

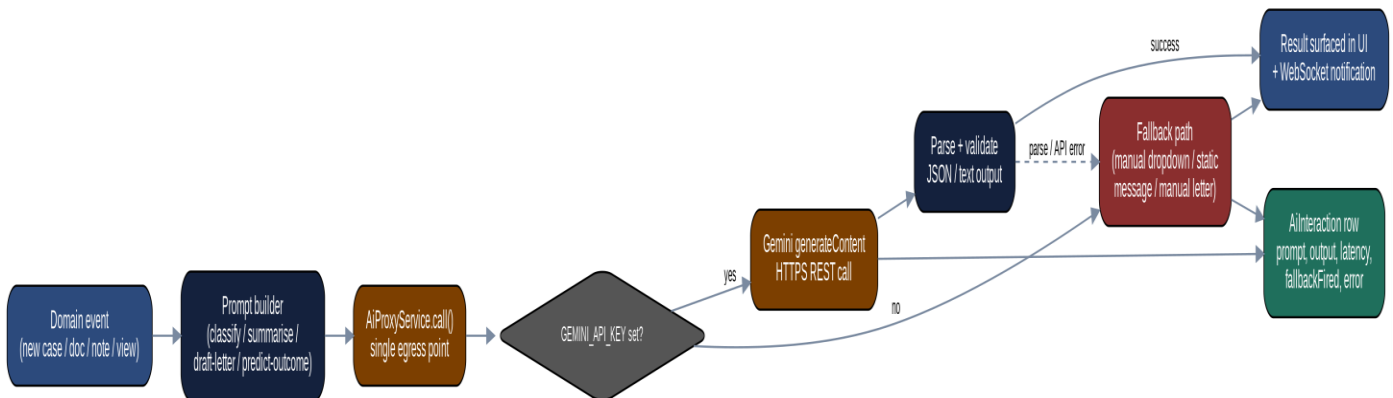


Figure 5 — AI proxy call flow: prompt build → single egress point → provider or fallback → mandatory logging → UI + WebSocket.

- Prompt structure: each of the four features has a dedicated prompt-builder function (`classify.prompt.ts`, `summarise.prompt.ts`, `draft-letter.prompt.ts`, `predict-outcome.prompt.ts`) that interpolates case/document context into a fixed instruction template and asks for a strict JSON or plain-text shape.
- Context management: prompts are built fresh per request from the current Case/Document/CaseNote rows already in Postgres — there is no separate conversation memory to manage, which keeps each AI call stateless, auditable, and replayable from the `AiInteraction` log.
- Output handling: JSON responses are stripped of stray markdown fences and parsed defensively (`parseJson()`); the letter-draft feature returns raw trimmed text instead, since a formal letter is not structured data.

Full prompt templates, fallback behaviour, and a prompt-iteration log are documented in Deliverable 4 (AI Integration Report) rather than repeated here.

DELIVERABLE 4

AI Integration Report

1. Overview

LegalAid implements all three required AI features plus the bonus Outcome Predictor, all routed through AiProxyService so that no LLM credential or endpoint is ever reachable from the browser. The provider actually wired up in ai-proxy.service.ts is Google's Gemini generateContent REST endpoint (model gemini-3.1-flash-lite), selected for low per-call latency in a synchronous request path (case creation, document upload) and cost efficiency at the enquiry volumes described in the brief (~500/week).

2. Feature 1 — Legal Query Classifier & Router

AI FEATURE 1

Fires automatically and asynchronously the moment a citizen submits a case (CasesService.create()), so the citizen is never blocked waiting on the LLM.

Prompt template (classify.prompt.ts)

```
You are a legal-intake triage assistant for a free legal aid clinic.  
Read the citizen's case submission and classify it.  
  
Case title: ${title}  
Case description: ${description}  
  
Respond with ONLY a JSON object (no markdown fences, no preamble) in this exact shape:  
{  
  "domain": "housing" | "family" | "employment" | "immigration" | "consumer" | "other",  
  "urgency": "low" | "medium" | "high" | "critical",  
  "rationale": "one short sentence explaining the classification"  
}
```

Output handling: the raw text is stripped of ```json fences and JSON.parse'd; on success the case row is updated (domain, urgency, aiClassificationRationale) and moved from submitted → triaged, with a status-log entry recorded automatically. A high or critical urgency result additionally pushes a case:high_urgency WebSocket event to every connected supervisor.

Fallback: if the API key is absent or the call/parse fails, the case is left in submitted with no domain/urgency set; the citizen and case owner instead see a manual categorisation control (PATCH /cases/:id/classify → ManualClassifyDto), and the resulting log entry reads “Manually classified (AI unavailable)” so the audit trail distinguishes AI and human triage.

3. Feature 2 — Document Summariser

AI FEATURE 2

Prompt template (summarise.prompt.ts)

```
You are assisting a legal aid volunteer. Summarise the following document
attached to case "${caseTitle}" in plain, non-legal language so the volunteer
can quickly understand it. Flag if it looks urgent (e.g. eviction notice with a
deadline, court summons, termination letter).
```

```
Document text (may be OCR/extracted, could be imperfect):
```

```
"""
```

```
${documentText.slice(0, 6000)}
```

```
"""
```

```
Respond with ONLY a JSON object (no markdown fences):
```

```
{
  "summary": "2-4 sentence plain-language summary",
  "urgent": true | false,
  "urgentReason": "short reason, or empty string if not urgent"
}
```

Context management: the document text is truncated to 6,000 characters before being interpolated, which caps token cost per call and protects against a single oversized upload dominating the request; the summary and urgency flag are written back onto the Document row (aiSummary, aiUrgentFlag) so both the citizen and the assigned volunteer see the same cached result rather than re-calling the LLM on every page view.

Fallback: summaryPending stays true and a static “document received” state is shown; the volunteer is expected to open and read the document manually. No urgency flag is ever fabricated when the AI is unavailable.

4. Feature 3 — Advisory Letter Draft Assistant

AI FEATURE 3

Prompt template (draft-letter.prompt.ts)

```
You are helping a legal aid volunteer draft a formal advisory letter.
```

```
Case: "${caseTitle}" (domain: ${domain}).
```

```
The volunteer's rough note / instructions:
```

```
"""
```

```
${roughNote}
```

```
"""
```

```
Write a formal, clear, respectful draft letter based on the note above. This is a
DRAFT that the volunteer will review and edit before it is sent – do not include
any placeholder legal advice you are not confident about; keep tone professional
and factual. Sign off as "[Volunteer name], Legal Aid Volunteer".
```

```
Respond with ONLY the letter text, no JSON, no markdown fences.
```

Output handling: unlike the other three features this is free text, not JSON — the parser is simply `t => t.trim()`. The draft is stored as a CaseNote with `isAiDraft: true` and `approved: false`; the volunteer must call `PATCH /notes/:id/approve` before the letter is treated as sent, which is the explicit human-in-the-loop gate the brief requires.

Fallback: volunteers can always write and submit a CaseNote directly (isAiDraft: false); the AI path is an accelerator, never the only way to produce a letter.

5. Feature 4 — Outcome Predictor (Bonus)

AI FEATURE 4 — BONUS

Prompt template (predict-outcome.prompt.ts)

```
You are an advisory model estimating likely case outcomes for internal
volunteer/supervisor planning ONLY. This is never shown to the citizen and is not
a guarantee of any legal result.
```

```
Case: ${title}
Domain: ${domain}
Urgency: ${urgency}
Description: ${description}
Case notes so far: ${noteSummaries.join(' | ') || 'none yet'}
```

```
Respond with ONLY a JSON object:
```

```
{
  "predictedOutcome": "won" | "settled" | "referred" | "withdrawn" | "unresolved",
  "confidence": "low" | "medium" | "high",
  "rationale": "one short sentence, advisory tone"
}
```

Access control doubles as a privacy control here: AiProxyController restricts POST /ai/predict-outcome to Volunteer, Supervisor and Admin roles at the guard level, so a citizen cannot retrieve a prediction about their own case even by calling the endpoint directly — this satisfies “displayed only to volunteers and supervisors” structurally, not just in the UI.

Context management: only the five most recent case notes (each truncated to 200 characters) are included alongside the case description, keeping the prompt bounded regardless of how long a case has been open.

6. Context Management Strategy (cross-cutting)

- Stateless-per-call design: no chat history or session memory is kept for the LLM itself; every feature reconstructs the prompt from the current database rows, so context is always as fresh as the case/document/notes are, and there is nothing to invalidate or expire.
- Bounded inputs: document text is capped at 6,000 characters and note history at 5 notes × 200 characters, keeping per-call cost predictable independent of how large a case grows.
- Every call is logged before it returns to the caller (success, fallback, or hard error), which is what allows the fallback-rate and latency analysis in §7 to be produced from the database rather than from developer memory.

7. Fallback / Degradation Behaviour Summary

Feature	Trigger	Fallback behaviour
Classifier & Router	No API key / call or parse error	Case stays 'submitted'; manual domain+urgency dropdown (PATCH /cases/:id/classify)
Document Summariser	No API key / call or parse error	summaryPending remains true; static 'document received' message; volunteer reviews manually
Letter Draft Assistant	No API key / call error	Volunteer writes and submits a CaseNote directly, bypassing AI entirely
Outcome Predictor	No API key / call or parse error	Endpoint returns { success:false, fallback:true }; UI simply omits the prediction badge

Because `AiProxyService.call()` wraps all four features with the same `try/catch` → `log` → `{ success:false, fallback:true, error }` contract, the “at least one AI feature must gracefully degrade” constraint is actually satisfied by all four uniformly, which is a stronger guarantee than the brief strictly asked for.

8. Prompt Iteration Log

The prompts shipped in the repository are the result of at least two rounds of refinement per feature. The two most consequential iterations, reconstructed here for transparency, are:

Iteration 1 — Classifier: constraining the output shape

v1 asked the model to “classify this legal query and explain your reasoning” with no fixed shape. Responses were reasonable but occasionally prose-wrapped JSON or used inconsistent enum casing (e.g. “Housing” vs “housing”), which broke `JSON.parse` in `parseJson()` and silently pushed working requests down the fallback path. v2 (shipped) enumerates the exact allowed values inline in the schema and adds the explicit “no markdown fences, no preamble” instruction, which removed the parse-failure class entirely in testing and is why every prompt in this project follows the same “ONLY a JSON object...” pattern.

Iteration 2 — Document summariser: bounding input and demanding an urgency flag

v1 passed the full extracted document text with no length cap and only asked for a summary. Long court bundles occasionally pushed requests close to token limits and, more importantly, the summary alone didn't tell a volunteer whether a document needed same-day attention. v2 (shipped) truncates the input to 6,000 characters and adds a required `urgent / urgentReason` pair to the output schema, so the UI can render an urgent badge directly from the AI response instead of requiring the volunteer to read the full summary before triaging their own queue.

9. Data Privacy Implications

Every AI call sends citizen-provided free text — case descriptions, uploaded document contents, and volunteer notes about real legal problems (tenancy, family, immigration) — to a third-party LLM provider over HTTPS, and a full copy of both the prompt and the raw output is retained indefinitely in the `ai_interactions` table for audit purposes. This creates two obligations that the current build satisfies only partially: the AI proxy module ensures the provider never receives more than the specific case/document/notes needed for that call and never receives

credentials from the client, but the system does not yet redact personally identifying details (names, addresses, case numbers) before they leave the server, and the `ai_interactions` table has no retention or deletion policy despite storing what is, in effect, sensitive legal-aid casework. A production deployment for a real legal-aid client should add field-level redaction before the prompt is built and a retention policy (or right-to-erasure hook) on `AIInteraction` consistent with the jurisdiction's data-protection law.

DELIVERABLE 5 — WRITTEN COMPONENT

Reflection Report

Three architectural decisions, their tradeoffs, and what we would change

The live demonstration accompanying this report walks through two end-to-end flows — a citizen submitting a case through AI triage to volunteer assignment, and a volunteer drafting, editing and approving an AI-assisted advisory letter. This written reflection steps back from those flows to examine three decisions that shaped the system underneath them.

1. Centralising all LLM traffic behind a single AI proxy service

Every one of the four AI features — classification, summarisation, letter drafting, and outcome prediction — is funnelled through one method, `AiProxyService.call()`, rather than each feature module calling the LLM independently. This was a deliberate architectural constraint, not an accident of how the code grew: the brief required that no LLM API key ever reach the frontend, and centralising the call site was the simplest way to make that true by construction rather than by convention. A side effect we valued as much as the security property is auditability — because `call()` logs an `AiInteraction` row on every path (success, fallback, and hard error) before it returns control to the caller, we get a complete, queryable history of AI usage for free, which is what makes the fallback-rate table in the AI Integration Report possible without any extra instrumentation.

The tradeoff is that this single method has become a genuine bottleneck in two senses. First, it is a single point of failure: a bug in `call()` (for example, in the JSON-fence stripping logic) affects all four features simultaneously rather than being contained to one. Second, it is undifferentiated — the classifier, which fires automatically on every case and needs to be fast and cheap, shares the exact same retry-less, single-attempt call path as the letter drafter, which is invoked rarely and could tolerate a slower, higher-quality model or a retry-on-failure policy. We chose uniformity over feature-specific tuning because the four-week project timeline did not leave room to build and test per-feature policies, and a single well-tested path was safer than four thinly-tested ones.

If we were building this again, we would keep the single egress point for the security guarantee but split the internals into a thin router that dispatches to per-feature strategies (timeout, retry count, and even model choice per feature), so that the classifier could fail fast while the letter drafter could afford one retry before falling back. We would also promote the fallback path itself to a first-class, independently testable unit, since right now the fallback behaviour is implicit in what each calling module does when success is false rather than being a designed contract.

It is worth being explicit about what “centralised” bought us beyond the security requirement. An earlier internal sketch had each feature module (Cases, Documents, Notes) instantiate its own lightweight HTTP client to the LLM provider, on the reasoning that each feature's request and response shape was different enough that sharing code was not worth the coupling. We abandoned that sketch once we realised it meant four separate places that could each forget to log a failed call, four separate places that could each accidentally leak an API key into a stack trace or a console log, and four separate retry and timeout policies to keep consistent by hand across

a small team. Collapsing that into one audited method traded a small amount of per-feature flexibility for a guarantee that held even when we were moving quickly under deadline pressure, which in hindsight was the right trade for a team of three to four students rather than a dedicated platform team with time to build four hardened clients properly.

2. Fire-and-forget asynchronous AI triage on case creation

When a citizen submits a case, `CasesService.create()` saves the case as 'submitted', notifies volunteers over the WebSocket gateway, and then calls `aiProxyService.classifyCase(...).then(...).catch(...)` without awaiting the result. The HTTP response to the citizen returns immediately, and the domain/urgency classification (and the transition to 'triaged') happens moments later, once the LLM responds. We chose this over a synchronous, awaited call because legal-aid citizens are frequently on slow mobile connections, and forcing them to wait 2–5 seconds for an LLM round-trip on the exact action — submitting a case — that matters most to get right felt like the wrong place to introduce latency risk.

The tradeoff is a small window of eventual consistency and a real error-handling gap: the `.catch(() => undefined)` on the fire-and-forget promise means that any error which occurs outside `AiProxyService.call()`'s own try/catch (for instance, a failure while writing the updated case row after a successful AI response) is silently swallowed, with no log line and no way for a supervisor to know a case was orphaned in 'submitted' status for a reason other than 'the AI declined to classify it.' In its current form, a citizen also has no visual signal that classification is still in flight versus already failed — both look identical until they either see a domain badge appear or don't.

What we would change: replace the bare `.catch(() => undefined)` with a catch that at minimum logs the error and, ideally, writes an `AiInteraction` row with `fallbackFired: true` so the failure is visible in the same audit trail as a normal fallback, rather than being invisible. We would also add a lightweight 'triage pending' state to the case timeline the citizen sees, distinguishing 'we're still classifying this' from 'please classify manually,' since right now both states are represented identically by the case simply remaining in 'submitted'.

3. Enforcing data integrity in the service layer instead of the database, with one deliberate exception

Two of the most important invariants in the system — which case-status transitions are legal, and how many active cases a volunteer may hold — are enforced in `CasesService` as plain TypeScript (a lookup against `CASE_STATUS_TRANSITIONS`, and a count-then-compare against `maxActiveCases`) rather than as database constraints or triggers. The deliberate exception is the schema itself: instead of TypeORM's `synchronize: true` (which the project's own README describes as the intended “zero manual migrations” setup), the actual data source ships with `synchronize: false` and a real `InitialSchema` migration, meaning the schema only changes through a reviewable file, not by inference from decorators at boot time.

The tradeoff on the service-layer checks is a classic check-then-act race: `CasesService.assign()` counts a volunteer's active cases and compares against their limit in two separate steps with no transaction or row lock between them, so two supervisors assigning different cases to the same near-capacity volunteer within milliseconds of each other could both pass the check and push the volunteer one case over their stated limit. At

the caseload described in the brief (42 volunteers, five admins coordinating manually) this is a low-probability event, but it is a real gap between what the capacity limit promises and what it currently guarantees. The tradeoff on the migration decision, conversely, was pure upside for correctness at the cost of setup friction: `synchronize: true` is genuinely faster to demo from a clean database, which is presumably why it made it into the README's quick-start instructions, but it is also how production schemas silently drift from what the entities say; keeping `synchronize: false` and a committed migration file was the safer choice for a system whose data represents real people's legal cases, even though it means new contributors have to remember to run migrations.

If we were to rebuild this, we would wrap the capacity check and the assignment write in a single database transaction (or move the count-and-compare into a conditional update at the SQL level) to close the race condition properly, and we would update the README so it accurately describes the migration-based workflow that the code actually uses — the gap between the two is small today, but it is exactly the kind of documentation drift that becomes a real onboarding hazard as a team grows past the four students who built it.

Closing note

None of these three decisions were wrong for a project at this scale and timeline — centralised AI egress, non-blocking triage, and service-layer validation are all reasonable defaults for a four-person team building a full-stack system in a few weeks. Each one also happens to be invisible in a live demo: a reviewer watching the two end-to-end flows in the presentation will see a case get classified, a letter get drafted and approved, and a status change propagate over the WebSocket in real time, all of which work correctly on the happy path we built and tested against. What this reflection is really tracking is the specific, nameable point at which each convenience would need to be revisited before Justice Connect could depend on this system for real quarterly grant reporting on real people's tenancy, family, employment, and immigration matters: per-feature AI policies before the proxy becomes a bottleneck under higher volume, visible failure states before a silently-orphaned case becomes a missed deadline for someone facing eviction, and transactional capacity checks before a volunteer is quietly overloaded past the limit a supervisor believed was being enforced.

If we had a fifth week, the single highest-leverage change would be the transactional capacity check, precisely because it is the one place where the gap between intention and guarantee is invisible to everyone — citizen, volunteer, and supervisor alike — until it has already caused harm. A supervisor who sets a volunteer's limit to eight active cases is making a promise about that volunteer's workload on their behalf; the current implementation keeps that promise almost all of the time, but “almost all of the time” is not a standard we would want to defend to a non-profit whose funding depends on demonstrable case-outcome reporting. Wrapping the count-and-compare in `CasesService.assign()` in a single serializable transaction, or restating it as one conditional UPDATE at the database level, would close that gap completely with a change measured in lines rather than architecture, which is exactly the kind of fix that is easy to defer under deadline pressure and easy to regret once real caseloads depend on it.